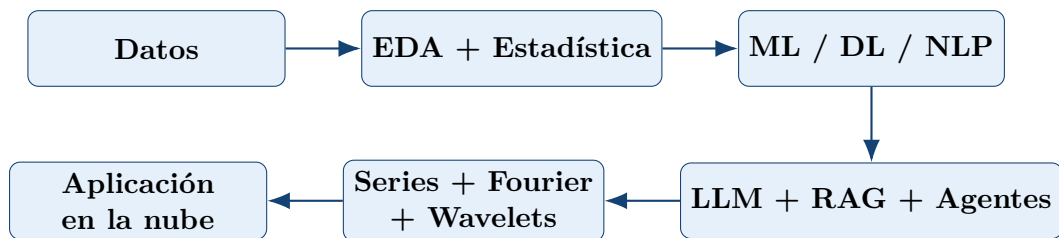


PROYECTO FINAL

Diplomado de Python y Análisis de Datos

Universidad Marista

Construcción de una solución de datos e
inteligencia artificial desplegada
completamente en la nube



Modalidad: Proyecto integrador

Producto final: Aplicación funcional + despliegue + repositorio + documentación + demo

Índice

1. Propósito del proyecto final	3
2. Regla principal: integración real	3
3. Requisitos obligatorios por bloque del diplomado	4
3.1. Bloque A — Python, datos y Pandas	4
3.2. Bloque B — EDA y visualización	4
3.3. Bloque C — Estadística	5
3.4. Bloque D — Series de tiempo	5
3.5. Bloque E — Feature Engineering	6
3.6. Bloque F — Machine Learning supervisado	6
3.7. Bloque G — Machine Learning no supervisado	7
3.8. Bloque H — Pipelines, validación y ensembles	7
3.9. Bloque I — Deep Learning	7
3.10. Bloque J — NLP clásico	8
3.11. Bloque K — Embeddings y Transformers	8
3.12. Bloque L — LLM	9
3.13. Bloque M — RAG	9
3.14. Bloque N — Agentes y Tool Calling	10
3.15. Bloque O — Fourier y análisis espectral	11
3.16. Bloque P — Wavelets y análisis multiresolución	11
4. Arquitectura mínima esperada	12
5. Despliegue obligatorio en la nube	12
5.1. Requisitos de despliegue	13
6. Interfaz y experiencia de usuario	13
7. Repositorio	14
7.1. README obligatorio	14
8. Entregables finales	15
9. Tabla de trazabilidad obligatoria	15
10. Reporte técnico	16
11. Presentación final	17
12. Rúbrica de evaluación — 100 puntos	17
13. Penalizaciones	18
14. Criterios de excelencia	19
15. Ideas de proyectos	20
15.1. 1. Plataforma inteligente de monitoreo industrial	20
15.2. 2. Inteligencia financiera	20

15.3. 3. Analítica inteligente de ventas	20
15.4. 4. Sistema inteligente para educación	21
15.5. 5. Plataforma de análisis de salud o bienestar con datos no sensibles/simulados	21
15.6. 6. Centro inteligente de soporte	21
16.Cronograma sugerido	22
17.Checklist final	22
18.Regla final	23

1. Propósito del proyecto final

Objetivo

El proyecto final deberá demostrar que el equipo es capaz de diseñar, construir, evaluar y publicar una solución completa de análisis de datos e inteligencia artificial utilizando de manera integrada los conocimientos desarrollados durante el diplomado.

El objetivo no es entregar una colección de notebooks aislados ni una demostración local, sino construir un **producto funcional** que resuelva un problema concreto y que pueda ser utilizado desde Internet.

El equipo podrá construir, por ejemplo:

- una aplicación web completa;
- un dashboard interactivo con Streamlit;
- una aplicación con FastAPI/Flask/Django y frontend web;
- una plataforma analítica;
- un sistema inteligente de monitoreo;
- un sistema de recomendación;
- una aplicación de predicción;
- un asistente inteligente con RAG;
- un sistema de detección de anomalías;
- una plataforma de análisis financiero;
- un sistema de análisis de sensores;
- una solución de NLP;
- una aplicación basada en agentes;
- una solución híbrida que combine varias de las anteriores.

Importante

La aplicación deberá estar **desplegada en la nube y accesible mediante una URL funcional**. Una solución que únicamente funcione en localhost, Jupyter Notebook o la computadora del equipo **no se considerará una entrega completa**.

2. Regla principal: integración real

El proyecto deberá integrar **todo el recorrido del diplomado**. Sin embargo, integrar no significa agregar tecnologías solo para mencionarlas.

Cada componente deberá:

1. resolver una necesidad real del proyecto;

2. estar conectado con el flujo general del sistema;
3. tener una justificación técnica;
4. producir un resultado observable;
5. incluir una métrica, prueba o evidencia cuando corresponda;
6. documentarse en el README y en el reporte técnico.

Atención

No será suficiente escribir en el README frases como “se utilizó Machine Learning”, “se utilizó RAG” o “se utilizaron Wavelets”. Se deberá mostrar **dónde, por qué, cómo, con qué datos y con qué resultado** se aplicó cada técnica.

3. Requisitos obligatorios por bloque del diplomado

3.1. Bloque A — Python, datos y Pandas

El proyecto deberá incluir:

- carga de datos desde CSV, API, base de datos, archivos o fuente equivalente;
- uso de Pandas;
- inspección de estructura y tipos;
- tratamiento de valores faltantes;
- detección y manejo de duplicados;
- tratamiento razonado de valores atípicos;
- transformación de variables;
- combinación de fuentes mediante `merge`, `join` o proceso equivalente;
- creación de un dataset final reproducible.

Entregable obligatorio

Debe existir dentro del sistema o reporte una sección titulada **Pipeline de Datos**, donde se explique desde dónde llegan los datos y cómo se transforman hasta quedar listos para análisis/modelado.

3.2. Bloque B — EDA y visualización

Se deberá realizar un análisis exploratorio que incluya:

- estadísticas descriptivas;
- distribuciones;
- relaciones entre variables;

- segmentaciones relevantes;
- identificación de patrones;
- visualizaciones interpretables;
- conclusiones derivadas de los datos.

La aplicación deberá contener un **dashboard** o sección visual donde el usuario pueda consultar información relevante.

Las gráficas deberán ser funcionales, etiquetadas y aportar información al problema.

3.3. Bloque C — Estadística

El proyecto deberá aplicar conceptos estadísticos pertinentes al problema.

Al menos deberá incluir una combinación razonada de:

- media, mediana, varianza y desviación estándar;
- intervalos de confianza;
- correlación;
- comparación de grupos;
- prueba de hipótesis;
- interpretación de valor p cuando corresponda;
- distribuciones;
- análisis de incertidumbre.

Importante

La estadística deberá utilizarse para responder una pregunta del proyecto. No se aceptará ejecutar una prueba estadística sin explicar qué hipótesis se está evaluando y qué significa el resultado.

3.4. Bloque D — Series de tiempo

El sistema deberá trabajar con al menos una variable temporal o construir un módulo de análisis temporal.

Deberá incluir, según el problema:

- orden temporal correcto;
- análisis de tendencia;
- estacionalidad;
- ventanas móviles;
- rezagos;

- cambios de régimen;
- predicción temporal o análisis de ciclos.

3.5. Bloque E — Feature Engineering

Se deberán construir características derivadas útiles para el modelado.

Ejemplos:

- variables temporales;
- lags;
- rolling statistics;
- ratios;
- agregaciones;
- codificación de categorías;
- variables extraídas de texto;
- energía de Wavelets;
- componentes espectrales;
- embeddings.

El reporte deberá justificar por qué cada grupo de features puede aportar información.

3.6. Bloque F — Machine Learning supervisado

El proyecto deberá incluir **OBLIGATORIO** al menos un problema supervisado, siempre que la naturaleza del proyecto lo permita.

Podrá ser:

- regresión;
- clasificación;
- forecasting formulado como aprendizaje supervisado;
- detección supervisada de eventos.

Se deberá incluir:

- separación de entrenamiento y prueba;
- baseline;
- al menos dos modelos comparados;
- métricas apropiadas;
- interpretación de resultados;
- análisis de errores.

Ejemplos de métricas:

$$MAE, \quad RMSE, \quad R^2, \quad Accuracy, \quad Precision, \quad Recall, \quad F_1$$

3.7. Bloque G — Machine Learning no supervisado

Se deberá incluir al menos una aplicación de:

- clustering;
- PCA;
- detección de grupos;
- reducción de dimensionalidad;
- exploración de estructura latente.

El equipo deberá explicar qué utilidad tiene este análisis dentro del problema.

3.8. Bloque H — Pipelines, validación y ensembles

El proyecto deberá utilizar buenas prácticas de modelado:

- Pipeline de Scikit-learn o estructura equivalente;
- validación cruzada cuando sea apropiada;
- prevención de data leakage;
- comparación de modelos;
- al menos un método ensemble o una justificación técnica de por qué no es pertinente.

Ejemplos de ensemble:

- Random Forest;
- Gradient Boosting;
- XGBoost/LightGBM si se decide utilizarlos;
- Voting;
- Stacking.

3.9. Bloque I — Deep Learning

Deberá existir un componente de Deep Learning relacionado con el problema.

Ejemplos:

- MLP;
- red para clasificación;
- red para regresión;

- arquitectura aplicada a señales;
- representación aprendida;
- comparación contra un modelo clásico.

No se exige que Deep Learning sea el mejor modelo. Se exige:

implementarlo, evaluarlo y compararlo

3.10. Bloque J — NLP clásico

El sistema deberá incluir procesamiento de texto cuando el dominio lo permita.

Se pueden trabajar:

- limpieza de texto;
- tokenización;
- TF-IDF;
- análisis de sentimiento;
- clasificación de texto;
- extracción de palabras clave;
- similitud textual.

Si el proyecto originalmente no contiene texto, el equipo deberá diseñar una fuente textual pertinente, por ejemplo:

- comentarios;
- reportes;
- tickets;
- noticias;
- descripciones;
- manuales;
- incidencias.

3.11. Bloque K — Embeddings y Transformers

El proyecto deberá utilizar representaciones modernas de texto mediante:

- embeddings;
- Sentence Transformers;
- Transformers;
- similitud semántica;

- búsqueda semántica.

Deberá existir una comparación o explicación clara frente a una representación clásica como TF-IDF cuando corresponda.

3.12. Bloque L — LLM

La solución deberá integrar un modelo de lenguaje para una tarea real.

Ejemplos:

- resumen;
- explicación;
- clasificación;
- generación estructurada;
- consulta conversacional;
- generación de reportes;
- interpretación de resultados.

El equipo deberá controlar:

- prompt;
- formato de respuesta;
- alucinaciones;
- errores;
- validación de salida.

Importante

Si se utiliza Ollama, el modelo no podrá depender exclusivamente del equipo personal del estudiante. Deberá existir una estrategia desplegable en la nube: servidor/VM/contenedor con el modelo o un servicio equivalente accesible desde la aplicación publicada.

3.13. Bloque M — RAG

Deberá incluirse un módulo de **Retrieval-Augmented Generation**.

Mínimos:

1. documentos;
2. chunking;
3. embeddings;
4. índice/vector store o implementación equivalente;

5. consulta;
6. Top- k ;
7. construcción de contexto;
8. respuesta con LLM;
9. fuentes o metadatos;
10. comportamiento cuando no existe evidencia.

El equipo deberá evaluar al menos una métrica del Retriever:

$$Recall@k, \quad Precision@k, \quad MRR$$

o una métrica equivalente bien justificada.

3.14. Bloque N — Agentes y Tool Calling

La aplicación deberá incorporar un agente o router inteligente que pueda decidir entre herramientas.

Ejemplos de Tools:

- calculadora;
- predictor de ML;
- consulta a base de datos;
- RAG;
- analizador de series;
- módulo estadístico;
- API externa;
- generador de reporte.

Debe existir:

- definición clara de Tools;
- argumentos estructurados;
- validación;
- Dispatcher;
- logging;
- manejo de errores;
- límite de pasos;
- al menos una tarea multi-step.

Se deberá medir, en un conjunto de pruebas, al menos:

Tool Selection Accuracy

y reportar éxito de tareas.

3.15. Bloque O — Fourier y análisis espectral

El proyecto deberá incluir análisis frecuencial sobre una serie temporal o señal.

Se deberá mostrar:

- señal en dominio temporal;
- FFT;
- espectro;
- frecuencias dominantes;
- conversión frecuencia-periodo;
- interpretación de al menos un ciclo;
- filtrado o reconstrucción cuando sea pertinente.

Atención

Un pico espectral no se interpretará automáticamente como causalidad. Se deberá explicar únicamente qué periodicidad o componente sugiere.

3.16. Bloque P — Wavelets y análisis multiresolución

El proyecto deberá incluir una descomposición Wavelet con PyWavelets.

Mínimos:

- selección de Wavelet;
- DWT o CWT;
- aproximaciones;
- detalles;
- varios niveles;
- energía por nivel;
- reconstrucción;
- interpretación multiresolución.

Se valorará especialmente:

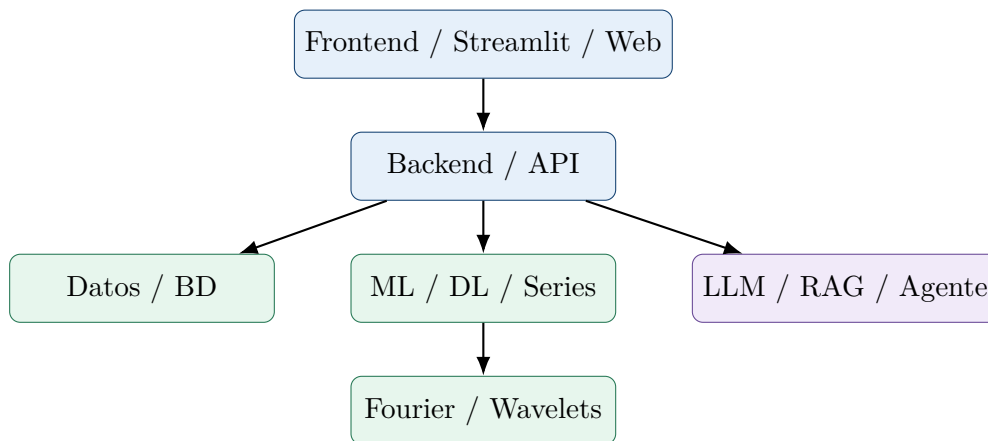
- detección de anomalías;
- denoising;

- generación de features Wavelet;
- comparación de señales;
- localización de eventos.

4. Arquitectura mínima esperada

El proyecto deberá presentar una arquitectura clara.

Una arquitectura posible es:



No es obligatorio utilizar exactamente esta arquitectura, pero la solución deberá ser modular y explicable.

5. Despliegue obligatorio en la nube

Importante

Toda la solución funcional deberá estar publicada. La evaluación se realizará utilizando la URL entregada, no ejecutando el proyecto desde la laptop de los alumnos.

Se podrán utilizar plataformas como:

- Streamlit Community Cloud;
- Render;
- Railway;
- Vercel;
- Netlify;
- Hugging Face Spaces;
- Google Cloud;
- AWS;
- Azure;

- Oracle Cloud;
- Supabase;
- Neon;
- MongoDB Atlas;
- servicios equivalentes.

La elección deberá depender de la arquitectura.

5.1. Requisitos de despliegue

1. URL pública funcional.
2. No depender de `localhost`.
3. Variables sensibles mediante secretos/variables de entorno.
4. No subir contraseñas o API keys al repositorio.
5. Dependencias reproducibles.
6. Archivo `requirements.txt`, `pyproject.toml` o equivalente.
7. Base de datos remota si el proyecto necesita persistencia.
8. Manejo de errores de red y servicios.
9. Tiempo de respuesta razonable.

6. Interfaz y experiencia de usuario

La aplicación deberá tener una interfaz utilizable por una persona que no haya desarrollado el proyecto.

Mínimos:

- página o pantalla inicial;
- descripción del problema;
- navegación clara;
- módulo de datos/EDA;
- módulo de modelos;
- módulo temporal/espectral;
- módulo de IA generativa;
- módulo de resultados;
- mensajes claros de error;
- identificación de fuentes en RAG;

- explicación de métricas.

7. Repositorio

El proyecto deberá estar en GitHub o repositorio equivalente.

Estructura sugerida:

```
proyecto-final/  
|  
|-- app/  
|-- src/  
|   |-- data/  
|   |-- preprocessing/  
|   |-- features/  
|   |-- ml/  
|   |-- deep_learning/  
|   |-- nlp/  
|   |-- rag/  
|   |-- agents/  
|   |-- signals/  
|  
|-- notebooks/  
|-- tests/  
|-- docs/  
|-- assets/  
|-- requirements.txt  
|-- README.md  
|-- .gitignore
```

7.1. README obligatorio

El README deberá incluir:

1. nombre del proyecto;
2. integrantes;
3. problema;
4. objetivo;
5. arquitectura;
6. stack tecnológico;
7. fuente de datos;
8. módulos;
9. modelos;
10. métricas;

11. Fourier/Wavelets;
12. LLM/RAG/Agentes;
13. instrucciones de desarrollo;
14. variables de entorno necesarias;
15. enlace a la aplicación desplegada;
16. capturas;
17. limitaciones;
18. trabajo futuro.

8. Entregables finales

Entregable obligatorio

La entrega final estará incompleta si falta cualquiera de los elementos principales siguientes.

1. **URL pública de la aplicación.**
2. **Repositorio completo.**
3. **README profesional.**
4. **Reporte técnico en PDF.**
5. **Presentación final.**
6. **Video demo** mostrando la aplicación desplegada.
7. **Dataset o instrucciones reproducibles** para obtenerlo.
8. **Modelo(s)** o mecanismo reproducible para obtenerlos.
9. **Resultados y métricas.**
10. **Diagrama de arquitectura.**
11. **Tabla de trazabilidad**, donde se indique en qué módulo se aplicó cada tema del diplomado.

9. Tabla de trazabilidad obligatoria

El reporte deberá contener una tabla similar a la siguiente:

Tema del diploma-	Dónde se aplicó	Evidencia / resultado
do		
CSV / Pandas	Pipeline de ingestión	Dataset procesado
EDA	Dashboard	Gráficas y hallazgos
Estadística	Comparación de grupos	IC / prueba / conclusión

Series temporales	Módulo temporal	Tendencia / estacionalidad
ML	Predictor	Métricas de test
Clustering/PCA	Exploración	Segmentos/componentes
DL	Modelo neuronal	Comparación contra base-line
NLP	Texto	Métrica / salida
Embeddings	Búsqueda semántica	Ranking / similitud
LLM	Asistente	Respuestas estructuradas
RAG	Consulta documental	Recall@k / fuentes
Agentes	Router/Tools	Tool accuracy
Fourier	Espectro	Periodos dominantes
Wavelets	Multiresolución	Energía / anomalías

10. Reporte técnico

El reporte deberá incluir como mínimo:

1. Resumen ejecutivo.
2. Problema.
3. Objetivo general.
4. Objetivos específicos.
5. Fuente y descripción de datos.
6. Arquitectura.
7. Ingeniería de datos.
8. EDA.
9. Estadística.
10. Series temporales.
11. Feature Engineering.
12. Machine Learning.
13. Deep Learning.
14. NLP y embeddings.
15. LLM.
16. RAG.
17. Agentes.
18. Fourier.
19. Wavelets.
20. Evaluación integral.

21. Despliegue.
22. Limitaciones.
23. Conclusiones.
24. Trabajo futuro.

11. Presentación final

La exposición deberá centrarse en el producto, no en leer definiciones.

Orden sugerido:

1. problema real;
2. usuario;
3. arquitectura;
4. datos;
5. hallazgos;
6. modelo;
7. inteligencia artificial;
8. series/señales;
9. demostración en vivo;
10. métricas;
11. conclusiones.

Importante

Durante la demostración se utilizará la **URL desplegada**. No se considerará equivalente abrir un notebook local o ejecutar `localhost`.

12. Rúbrica de evaluación — 100 puntos

Criterio	Puntos	Qué se evaluará
Problema, objetivos y valor de la solución	5	Claridad, utilidad y coherencia.
Calidad de datos, Pandas, limpieza, joins e ingeniería	7	Pipeline reproducible y correctamente documentado.
EDA y visualización	6	Gráficas útiles, interacción y conclusiones.

Criterio	Puntos	Qué se evaluará
Estadística	5	Uso correcto e interpretación.
Series temporales y Feature Engineering	6	Tratamiento temporal, lags, rolling o equivalentes.
Machine Learning supervisado	8	Baselines, modelos, métricas y análisis de error.
ML no supervisado, PCA y estructura latente	4	Aplicación útil e interpretación.
Pipelines, validación cruzada y ensembles	5	Buenas prácticas, prevención de leakage y comparación.
Deep Learning	5	Modelo funcional, evaluación y comparación.
NLP, embeddings y Transformers	6	Procesamiento textual y representación semántica.
LLM	5	Tarea útil, prompting, validación y manejo de errores.
RAG	8	Pipeline completo, fuentes y evaluación del Retriever.
Agentes / Tool Calling	7	Routing, tools, validación, multi-step y métricas.
Fourier / análisis espectral	5	FFT, espectro, ciclos e interpretación.
Wavelets / multiresolución	6	DWT/CWT, energía, reconstrucción e interpretación.
Arquitectura de software e integración	5	Modularidad y conexión real entre componentes.
Despliegue en la nube	5	URL estable, servicios remotos, manejo de secretos.
Documentación, README y repositorio	4	Orden, reproducibilidad, commits y claridad.
Demo y presentación final	3	Claridad, dominio y funcionamiento.
TOTAL	100	

13. Penalizaciones

Se podrán aplicar las siguientes penalizaciones:

Situación	Penalización orientativa
La aplicación no está desplegada públicamente	Hasta -30 puntos
La URL no funciona durante la evaluación	Hasta -15 puntos
Repositorio incompleto o no accesible	Hasta -10 puntos
API keys o contraseñas expuestas en GitHub	Hasta -15 puntos
No existe README reproducible	Hasta -8 puntos
Uso superficial de técnicas solo para “cumplir”	Hasta -20 puntos
Métricas inventadas o no reproducibles	Evaluación del apartado: 0
Plagio o entrega de código ajeno presentado como propio	Sujeto a normativa académica

14. Criterios de excelencia

Criterio de excelencia

Un proyecto sobresaliente no será el que utilice más librerías, sino el que conecte mejor los métodos con un problema real, mida correctamente sus resultados y entregue una experiencia funcional, reproducible y desplegada.

Características de un proyecto excelente:

- problema bien definido;
- datos reales o simulación científicamente justificada;
- arquitectura limpia;
- interfaz usable;
- modelos comparados;
- métricas correctas;
- interpretación crítica;
- RAG con fuentes;
- agentes con herramientas útiles;
- análisis temporal y espectral coherente;
- Wavelets utilizadas con propósito;

- despliegue estable;
- excelente documentación;
- demo convincente.

15. Ideas de proyectos

Estas ideas son únicamente referencias. Se pueden proponer otras.

15.1. 1. Plataforma inteligente de monitoreo industrial

- sensores;
- series de tiempo;
- detección de anomalías;
- Fourier;
- Wavelets;
- ML predictivo;
- RAG con manuales;
- agente para diagnóstico.

15.2. 2. Inteligencia financiera

- precios históricos;
- noticias;
- NLP;
- series;
- clasificación/regresión;
- Fourier;
- Wavelets;
- RAG sobre reportes;
- agente analista.

15.3. 3. Analítica inteligente de ventas

- clientes;
- productos;

- clustering;
- predicción;
- estacionalidad;
- recomendaciones;
- análisis de comentarios;
- RAG;
- asistente de negocio.

15.4. 4. Sistema inteligente para educación

- rendimiento;
- clustering de estudiantes;
- predicción de riesgo;
- NLP de respuestas;
- RAG de materiales;
- agente tutor;
- dashboard.

15.5. 5. Plataforma de análisis de salud o bienestar con datos no sensibles/simulados

- señales;
- series;
- clasificación;
- Fourier;
- Wavelets;
- detección de anomalías;
- explicación con LLM.

15.6. 6. Centro inteligente de soporte

- tickets;
- NLP;
- clasificación;
- clustering;

- embeddings;
- RAG;
- agente con Tool Calling;
- métricas de operación.

16. Cronograma sugerido

Etapa	Actividad
1	Definición del problema, usuario, datos y arquitectura.
2	Pipeline de datos, limpieza, EDA y estadística.
3	Series de tiempo, features, Fourier y Wavelets.
4	ML, clustering, PCA, ensembles y Deep Learning.
5	NLP, embeddings, LLM y RAG.
6	Agente, herramientas y tareas multi-step.
7	Integración frontend/backend y despliegue.
8	Evaluación, documentación, pruebas y demo final.

17. Checklist final

Antes de entregar, el equipo deberá poder contestar **sí** a todo lo siguiente:

- | | |
|--|--|
| ☐ ¿Existe una URL pública? | ☐ ¿Existe validación? |
| ☐ ¿La URL funciona? | ☐ ¿Existe ensemble? |
| ☐ ¿No depende de localhost? | ☐ ¿Existe Deep Learning? |
| ☐ ¿El repositorio está actualizado? | ☐ ¿Existe NLP? |
| ☐ ¿Existe README? | ☐ ¿Existen embeddings? |
| ☐ ¿Las API keys están protegidas? | ☐ ¿Existe LLM? |
| ☐ ¿Los datos son reproducibles? | ☐ ¿Existe RAG? |
| ☐ ¿Existe EDA? | ☐ ¿RAG muestra fuentes? |
| ☐ ¿Existe estadística? | ☐ ¿RAG fue evaluado? |
| ☐ ¿Existe análisis temporal? | ☐ ¿Existe agente? |
| ☐ ¿Existe Feature Engineering? | ☐ ¿Existen Tools? |
| ☐ ¿Existe ML supervisado? | ☐ ¿Existe tarea multi-step? |
| ☐ ¿Existe ML no supervisado? | ☐ ¿Existe FFT? |

- | | |
|--|---|
| ☐ ¿Existe análisis de frecuencia? | ☐ ¿Existe reporte técnico? |
| ☐ ¿Existe Wavelet? | ☐ ¿Existe video demo? |
| ☐ ¿Existen aproximaciones/detalles? | ☐ ¿Existe presentación? |
| ☐ ¿Existe evaluación por nivel? | ☐ ¿Existe tabla de trazabilidad? |
| ☐ ¿Existe diagrama de arquitectura? | |

18. Regla final

El proyecto final deberá demostrar que los contenidos del diplomado pueden convertirse en una solución real, integrada, medible y disponible en Internet.

No se evaluará únicamente que el código “corra”.

Se evaluará que el equipo pueda explicar:

qué hizo + por qué lo hizo + cómo lo midió + qué valor produce

¡Éxito con su proyecto final!